Null Return

DZAP

SMART CONTRACT AUDIT REPORT

Produced for **DZap**
by **Null Return**

Date: **June 2025**

PUBLIC

Contents

Prepared by

Project Manager: Nataly Demidova **Email**: nataly@nullreturn.io

Lead Auditor: Alexander Mazaletskiy **Email**: am@nullreturn.io

Introduction

This document presents the results of a smart contract audit conducted by Null Return for the DZap protocol. The purpose of the audit was to evaluate the security and correctness of DZap's smart contracts prior to deployment. Special attention was given to identifying vulnerabilities that may lead to loss of funds, unauthorized contract interactions, or logical errors in fee distribution, wallet execution, and cross-protocol zapping operations.

About DZap

DZap is a cross-protocol zapping platform designed to facilitate seamless token swaps, liquidity provisioning, and wallet-based transaction execution. It enables users to perform complex operations (e.g., token transfers, approvals, and external contract interactions) in a single transaction, leveraging a modular architecture with secure backend validation. Key features include:

- **Zapping Operations**: Aggregates multiple token interactions (e.g., ERC20 transfers, approvals) into a single transaction for efficiency.
- **Wallet Management**: Supports user-controlled smart wallets with validator-based execution and nonce-protected signatures.
- **Fee and Referral System**: Implements a flexible fee structure with referral incentives, secured by backend checks.
- **Backend Security**: Relies on a trusted backend for data signing, whitelisting, and validation to mitigate on-chain risks.
- **Permit and Permit2 Integration**: Supports gas-efficient token approvals via permit mechanisms.

The contracts are designed for interoperability with various DeFi protocols, with a focus on security, gas efficiency, and user experience.

Project Overview

DZap is designed as a flexible zapping platform built to serve users looking to execute complex DeFi operations efficiently. By aggregating token swaps, approvals, and external contract calls into a single transaction, it reduces gas costs and improves user experience. The protocol supports smart wallets with validator-based execution, enabling secure and customizable transaction flows. Its fee and referral system incentivizes participation while maintaining economic security through backend validation. The architecture relies on a combination of on-chain logic and off-chain backend controls, ensuring robust security for whitelisted adapters, signed data, and user interactions. The use of permit and Permit2 mechanisms enhances gas efficiency for token approvals.

Scope

This audit covered the core components of the DZap protocol, with a focus on the correctness, safety, and robustness of its smart contracts. The analysis was based on the source code provided in the DZapIO/DZapZappingContracts repository at the specified commit.

Field	Details
Repository	https://github.com/DZapIO/DZapZappingContracts
Initial Commit	ce086777236814235275057 23c9fda532c60728d
Reaudited Commit 1	d1def85b720df5cde679bcd36ffef8d1dbc37c33
Reaudited Commit 2	fec133a4bc89bcbeec2a6070696a949a7e634eb3
Scope 1	DZapZappingContracts/contracts/zap/DZapZapCore.sol
Scope 2	DZapZappingContracts/contracts/wallet/DZapWalletFactory.sol
Scope 3	DZapZappingContracts/contracts/wallet/DZapWallet.sol
Scope 4	DZapZappingContracts/contracts/wallet/DZapWalletManager.sol
Scope 5	DZapZappingContracts/contracts/wallet/DZapExecutor.sol
Scope 6	DZapZappingContracts/contracts/shared/libraries/LibAsset.sol

Field	Details
Scope 7	DZapZappingContracts/contracts/shared/libraries/LibPermit.sol
Scope 8	DZapZappingContracts/contracts/shared/libraries/LibValidator.sol

Methodology

The goal of this audit was to identify potential vulnerabilities, logical errors, and deviations from best practices in the DZap smart contracts. Our process followed a structured and layered approach, including:

1. **Manual Code Review** Each file in scope was reviewed manually by experienced security auditors to analyze:
    - Correctness of zapping and wallet execution logic
    - Access control and validator permissions
    - Reentrancy, delegatecall, and frontrunning risks
    - Fee calculation and economic security
    - Backend integration assumptions
2. **Compliance Verification** The contracts were checked for conformance with relevant Ethereum standards, including:
    - ERC20 and Permit/Permit2 for token interactions
    - Non-standard wallet execution patterns
3. **Static Analysis & Tooling** We used automated tools (e.g., Slither, custom linters, and symbolic execution engines) to detect:
    - Known vulnerability patterns (e.g., delegatecall, reentrancy)
    - Unsafe arithmetic and overflow risks
    - Storage collisions and signature replay issues
4. **Threat Modeling** We examined critical assumptions and attack surfaces relevant to DZap's architecture, including:
    - Delegatecall risks in whitelisted adapters
    - Permit and Permit2 frontrunning vulnerabilities
    - Fee and referral system exploits
    - Backend dependency security
    - Wallet execution and validator compromise scenarios
5. **Issue Reporting & Validation** All issues were documented with severity levels (Low, Medium, High). After reporting, the DZap team responded and applied fixes, which we then re-reviewed and validated.

Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Findings

In this section, we document the security issues identified during the audit of DZap, along with their severity levels, status, and relevant recommendations. Each finding includes a description of the issue, potential impact, and suggestions for remediation.

ID	Title	Severity	Status
H-1	Absence of Signature Deadline Enables Exploitation of Future Contract Compromises	High	Resolved
H-2	Unrestricted Delegatecall Allows Complete Storage Manipulation	High	Acknowledged
H-3	Unrestricted Delegatecall Allows Complete Storage Manipulation in DZapWallet	High	Acknowledged
M-1	Permit Function Reverts on Maximum Allowance Attempts	Medium	Resolved

ID	Title	Severity	Status
M-2	Fee-on-Transfer Tokens Break Balance Validation Logic	Medium	Acknowledged
M-3	Unlimited Gas Consumption in Loop Operations	Medium	Acknowledged
M-4	Unrestricted External Calls Enable Arbitrary Contract Interaction	Medium	Acknowledged
M-5	Insufficient Fee Validation May Lead to Economic Exploits	Medium	Resolved
M-6	Missing Zero Address Validation in Critical Functions	Medium	Resolved
M-7	Potential Malicious Approvals Through Unvalidated Input Tokens	Medium	Acknowledged
M-8	Permit Frontrunning Causes Transaction Failures	Medium	Resolved
M-9	Permit2 Frontrunning Causes Transaction Failures	Medium	Resolved
L-1	Wallet Factory Logic Inconsistency with Address Validation	Low	No Issue

ID	Title	Severity	Status
L-2	Potential Self-Call Issues in Wallet Execution	Low	Resolved
L-3	Label Collision Possible in Wallet Factory	Low	Acknowledged
L-4	Missing Quorum Validation in Constructor	Low	Acknowledged
L-5	Insufficient Validation of dustReceiver Parameter	Low	Acknowledged
I-1	Missing Event Emission for Critical State Changes	Informational	Resolved
I-2	FeeVault Native ETH Handling Concerns	Informational	Acknowledged
I-3	Inefficient Linear Search in Validator Processing	Informational	Acknowledged
I-4	Inconsistent Error Handling Patterns	Informational	Resolved

High

[H-1] Absence of Signature Deadline Enables Exploitation of Future Contract Compromises

- **Status:** Resolved
- **Location:** `DZapZapCore.sol#173`
- **Impact:** Valid signatures remain exploitable indefinitely if any target contract in the zap operation becomes compromised in the future. Attackers could use old, valid signatures to interact with hacked contracts, potentially draining user funds.

- **Description:** While `_handleVerification` uses nonces to prevent replay attacks, it lacked deadline validation unlike `DZapWallet`. This meant signatures remained valid forever, allowing attackers to exploit old signatures if a previously safe contract becomes compromised.
- **Proof of Concept:**

```
// Day 1: User signs transaction to interact with safe
 ↪  contract A
// Day 30: Contract A gets compromised/hacked
// Day 31: Attacker uses the old signature to force user
 ↪  interaction with now-malicious contract A
// Result: User funds drained through compromised contract
```

- **Recommendation:** Add deadline validation consistent with `DZapWallet`:

```
function _handleVerification(bytes32 _transactionId, address
 ↪  _referral, uint256 _deadline, bytes calldata _data, bytes
 ↪  calldata _signature) private {
    require(_deadline >= block.timestamp, "Signature
     ↪  expired");
    // existing verification logic
}
```

- **Client Response:** Added deadline validation for zap operations in commit d1def85.

[H-2] Unrestricted Delegatecall Allows Complete Storage Manipulation

- **Status:** Acknowledged
- **Location:** `DZapZapCore.sol#198-203`
- **Impact:** Malicious contracts in the whitelist can execute arbitrary code in the context of DZapZapCore, potentially modifying critical storage variables like owner, feeVault, or allowedCalls mapping, leading to complete protocol compromise.
- **Description:** The `_execute` function performs delegatecall to whitelisted addresses without restricting the callable functions or protecting critical storage slots. Since delegatecall executes external code with the caller's storage context, any whitelisted malicious contract can modify the core contract's state.
- **Proof of Concept:**

```
contract MaliciousContract {
    function exploit() external {
        // This executes in DZapZapCore's context
```

```
        // Can modify owner, feeVault, etc.
        // Storage layout manipulation possible
    }
}
```

- **Recommendation:**
  1. Implement function selector whitelisting for delegatecalls.
  2. Or completely remove delegatecall functionality.
- **Client Response:** Delegatecall is restricted to whitelisted adapters deployed and managed by our secure backend infrastructure, ensuring no issues arise. Our backend employs secure key management, regular audits, and role-based access controls to prevent unauthorized adapter deployment.

[H-3] Unrestricted Delegatecall Allows Complete Storage Manipulation in DZapWallet

- **Status:** Acknowledged
- **Location:** `DZapWallet.sol#77-91`
- **Impact:** Validators can execute delegatecalls to any contract, allowing malicious or compromised contracts to rewrite critical wallet storage, including owner, verifier, and whitelist mappings, resulting in complete wallet takeover and fund theft.
- **Description:** The `execute` function allows validators to perform delegatecall operations without restrictions on target contracts or callable functions, enabling storage manipulation through delegatecall context.
- **Proof of Concept:**

```
contract MaliciousStorageRewriter {
    function innocentFunction() external {
        // Appears harmless but rewrites critical storage
        assembly {
            // Directly overwrite storage slots
            sstore(0, caller())                    // Hijack
             ↪ owner (slot 0)
            sstore(1,
             ↪ 0x0000000000000000000000000000000000000000)
             ↪ // Clear verifier (slot 1)
            mstore(0x00, caller())                 // key:
             ↪ attacker address
            mstore(0x20, 2)                        // slot:
             ↪ whitelist mapping at slot 2
```

```
            let slot := keccak256(0x00, 0x40)
            sstore(slot, 1)                          //
             ↪  whitelist[attacker] = true
        }
    }

    function stealFunds(address token) external {
        // Now can drain all funds as new owner
        IERC20(token).transfer(msg.sender,
         ↪  IERC20(token).balanceOf(address(this)));
    }
}
```

- **Recommendation:**
  1. Implement function selector whitelisting for delegatecalls.
  2. Or completely remove delegatecall functionality.
- **Client Response:** Delegatecall is restricted to whitelisted adapters deployed and managed by our secure backend infrastructure, preventing issues. Our backend uses multi-signature wallets, encrypted communication, and regular security audits to ensure adapter integrity.

Medium

[M-1] Permit Function Reverts on Maximum Allowance Attempts

- **Status:** Resolved
- **Location:** `LibPermit.sol#25`
- **Impact:** Users cannot set maximum allowance (`type(uint256).max`) through the permit function, causing transaction failures and breaking expected functionality for protocols that commonly use max allowances.
- **Description:** The `permit` function performs `_amount + 1` without handling the edge case where `_amount` equals `type(uint256).max`, causing an overflow revert.
- **Proof of Concept:**

```
// This will revert due to overflow protection
uint256 maxAmount = type(uint256).max; // 2^256 - 1
// _amount + 1 causes arithmetic overflow and reverts
IERC20Permit(_token).permit(_from, _to, maxAmount + 1,
 ↪  deadline, v, r, s);
```

- **Recommendation:** Handle the maximum allowance case explicitly:

```
uint256 permitAmount = _amount == type(uint256).max ?
 ↪  type(uint256).max : _amount + 1;
IERC20Permit(_token).permit(_from, _to, permitAmount,
 ↪  deadline, v, r, s);
```

- **Client Response:** Changed `_amount + 1` to `_amount` in commit d1def85.

[M-2] Fee-on-Transfer Tokens Break Balance Validation Logic

- **Status:** Acknowledged
- **Location:** `LibAsset.sol#75-79`
- **Impact:** The protocol becomes incompatible with fee-on-transfer tokens, causing transactions with such tokens to revert and blocking their use.
- **Description:** The `transferFromERC20` function enforces strict balance equality check, which fails for tokens that charge fees during transfer.
- **Proof of Concept:**

```
// Token charges 0.1% fee on transfer
transferFromERC20(feeToken, user, contract, 1000);
// Actual received: 999, expected: 1000
// Transaction reverts with InvalidAmount()
```

- **Recommendation:** Return the actual transferred amount and handle fee-on-transfer tokens:

```
function transferFromERC20(address _token, address _from,
 ↪  address _to, uint256 _amount) internal returns (uint256
 ↪  actualAmount) {
    IERC20 token = IERC20(_token);
    uint256 prevBalance = token.balanceOf(_to);
    SafeERC20.safeTransferFrom(token, _from, _to, _amount);
    actualAmount = token.balanceOf(_to) - prevBalance;
    require(actualAmount > 0, InvalidAmount());
}
```

- **Client Response:** We do not support tokens that charge fees during transfer.

[M-3] Unlimited Gas Consumption in Loop Operations

- **Status:** Acknowledged

- **Location:** DZapZapCore.sol#381-386
- **Impact:** Large arrays in _handleErcDeposits and similar loop operations can cause transactions to run out of gas, leading to failed transactions and poor user experience.
- **Description:** Unbounded loops process user-provided arrays without gas limit considerations, potentially causing gas exhaustion.
- **Recommendation:** Implement reasonable limits on array sizes:

```
uint256 constant MAX_INPUT_TOKENS = 50;
require(_inputTokens.length <= MAX_INPUT_TOKENS, "Too many
 ↪  input tokens");
```

- **Client Response:** These cases are handled at the backend level, with secure validation to limit array sizes and prevent gas exhaustion.

[M-4] Unrestricted External Calls Enable Arbitrary Contract Interaction

- **Status:** Acknowledged
- **Location:** DZapZapCore.sol#204-206
- **Impact:** Non-delegatecall external calls are not restricted by whitelisting, allowing users to call arbitrary contracts and potentially exploit integrations or perform unintended token approvals.
- **Description:** Regular external calls in the _execute function have no restrictions, allowing calls to any contract with any data.
- **Proof of Concept:**

```
// User can call any contract through zapData
zapData.callTo = maliciousContract;
zapData.callData = abi.encodeWithSignature("exploit()");
zapData.isDelegateCall = false; // Bypasses whitelist check
```

- **Recommendation:** Implement whitelisting for all external calls or restrict callable functions:

```
if (_zapData.callData.length != 0) {
    require(allowedCalls[_zapData.callTo],
     ↪  UnauthorizedCall(_zapData.callTo));
    if (_zapData.isDelegateCall) {
        (success, res) =
 ↪  _zapData.callTo.delegatecall(_zapData.callData);
```

```
    } else {
        (success, res) = _zapData.callTo.call{ value:
↪   _zapData.nativeValue }(_zapData.callData);
    }
    require(success, CallFailed(res));
}
```

- **Client Response:** Risks are mitigated with two-level backend validation (whitelisting + secure data signing) using encrypted communication and multi-signature approvals, though this limits on-chain flexibility and requires trusted backend infrastructure.

[M-5] Insufficient Fee Validation May Lead to Economic Exploits

- **Status:** Resolved
- **Location:** DZapZapCore.sol#191-193
- **Impact:** Unchecked fee parameters could result in integer overflow or fees above 100%, potentially draining user funds or breaking fee distribution logic.
- **Description:** The `_getTotalAnReferralFeeAmount` function performs multiplication without validating that fees represent valid percentages or checking for overflow.
- **Proof of Concept:**

```
uint256 amount = type(uint256).max / 2;
uint256 fee = _BPS_DENOMINATOR; // 100%
// FullMath.mulDiv could overflow or produce unexpected
↪  results
```

- **Recommendation:** Add proper fee validation and overflow protection:

```
function _getTotalAnReferralFeeAmount(uint256 _amount, uint256
↪  _fee, uint256 _referralFee) private pure returns (uint256
↪  totalFeeAmount, uint256 referralFeeAmount) {
    require(_fee <= _BPS_DENOMINATOR, "Fee exceeds 100%");
    require(_referralFee <= _BPS_DENOMINATOR, "Referral fee
     ↪  exceeds 100%");

    totalFeeAmount = FullMath.mulDiv(_amount, _fee,
↪   _BPS_DENOMINATOR);
    if (_referralFee != 0) {
        referralFeeAmount = FullMath.mulDiv(totalFeeAmount,
↪   _referralFee, _BPS_DENOMINATOR);
    }
```

```
}
```

- **Client Response:** Referral fees are ensured to never exceed `_BPS_DENOMINATOR` via checks in `addReferral()` and `setDefaultReferralFee()`. Added check for token fee to ensure it is less than `MAX_TOKEN_FEE` in commit d1def85.

[M-6] Missing Zero Address Validation in Critical Functions

- **Status:** Resolved
- **Location:** `DZapExecutor.sol#92-96`
- **Impact:** Zero addresses could be added to the executor whitelist, potentially causing authorization issues or unexpected behavior.
- **Description:** The `_setExecutors` function doesn't validate against zero addresses when updating executor permissions.
- **Recommendation:** Add zero address validation:

```
function _setExecutors(address[] memory _executorsArr, bool
 ↪  _whitelisted) private {
    uint256 length = _executorsArr.length;
    for (uint256 i; i < length; ++i) {
        require(_executorsArr[i] != address(0), ZeroAddress());
        _executors[_executorsArr[i]] = _whitelisted;
    }
}
```

- **Client Response:** Added zero address validation check in commit d1def85.

[M-7] Potential Malicious Approvals Through Unvalidated Input Tokens

- **Status:** Acknowledged
- **Location:** `DZapZapCore.sol#270`
- **Impact:** Malicious users could craft input tokens that approve arbitrary spenders, potentially allowing unauthorized access to user funds or protocol assets.
- **Description:** The `_handleErc20Input` function approves `_inputToken.approveTo` without validating the address.
- **Recommendation:** Implement spender validation through a whitelist:

```
mapping(address => bool) public approvedSpenders;

modifier validSpender(address _spender) {
    require(approvedSpenders[_spender], "Unauthorized
     ↪  spender");
    _;
}

function _handleErc20Input(...) private
 ↪  validSpender(_inputToken.approveTo) {
    // existing logic
}
```

- **Client Response:** The entire zap data is created, verified, and signed by our secure backend, preventing manipulation by other parties. Backend security includes multi-signature wallets, encrypted data handling, and regular audits.

[M-8] Permit Frontrunning Causes Transaction Failures

- **Status:** Resolved
- **Location:** `LibAsset.sol#L116`
- **Impact:** Zap transactions can be frontrun and fail due to permit replay, causing transaction reverts and gas wastage, leading to poor UX and potential DoS scenarios.
- **Description:** The `depositERC20` function calls `permit()` without checking existing allowances, allowing attackers to frontrun and consume nonces.
- **Proof of Concept:**

```
// Attacker sees this in mempool and frontruns:
token.permit(user, spender, amount, deadline, v, r, s);

// User's original transaction fails:
token.permit(user, spender, amount, deadline, v, r, s); //
 ↪  Fails (nonce used)
// Entire zap transaction reverts
```

- **Indications to additional information:**
  - Lido Finance issue
- **Recommendation:** Check existing allowance before attempting permit:

```
function permit(address _token, address _from, address _to,
 ↪  uint256 _amount, bytes memory _data) internal {
    if (_data.length != 32 * 7) revert InvalidPermitData();
    (, , uint256 permitAmount, uint256 deadline, uint8 v,
 ↪  bytes32 r, bytes32 s) = abi.decode(
        _data,
        (address, address, uint256, uint256, uint8, bytes32,
 ↪  bytes32)
    );
    require(permitAmount >= _amount, "Permit amount
     ↪  insufficient");
    if (IERC20(_token).allowance(_from, _to) < _amount) {
        try IERC20Permit(_token).permit(_from, _to,
         ↪  permitAmount, deadline, v, r, s) {
        } catch {
            require(IERC20(_token).allowance(_from, _to) >=
             ↪  _amount, "Insufficient allowance");
        }
    }
}
```

- **Client Response:** We initially mitigated this by having our backend check the token's current allowance and only call permit if insufficient. To address the frontrunning issue where a consumed nonce causes a revert, we added try-catch logic to skip permit calls if the nonce is invalid, ensuring the transaction proceeds if sufficient allowance exists, as implemented in commit fec133a4.
- **Auditor Comment:** Without try-catch, a frontrun permit would cause a revert due to an invalid nonce. The try-catch approach is necessary to skip the permit call safely.

[M-9] Permute2 Frontrunning Causes Transaction Failures

- **Status:** Resolved
- **Location:** `LibAsset.sol#L114`
- **Impact**: Zap transactions using Permute2 can be frontrun and fail due to nonce consumption, causing transaction reverts and gas wastage, leading to poor UX and potential DoS scenarios.
- **Description:** The Permute2 integration calls `permit()` without nonce validation or allowance checks, allowing attackers to frontrun and consume nonces.
- **Proof of Concept:**

```
// User's transaction in mempool:
IPermit2.PermitSingle memory permitSingle =
 ↪  IPermit2.PermitSingle({
    details: IPermit2.PermitDetails({
        token: token,
        amount: amount,
        expiration: block.timestamp + 1800,
        nonce: 5 // User's current nonce
    }),
    spender: zapContract,
    sigDeadline: block.timestamp + 300
}
});

// Attacker frontruns:
permit2.permit(address(this), user, permitSingle, signature);
 ↪  // Consumes nonce 5

// User's transaction fails:
permit2.permit(address(this), user, permitSingle, signature);
 ↪  // Fails Reverts "InvalidNonce()"
```

- **Recommendation:** Implement defensive permit handling:

```
function safePermit2(
    address token,
    uint256 requiredAmount,
    IPermit2.PermitSingle memory permitSingle,
    bytes memory signature
    ) internal {
    (uint256 memory currentAllowance, uint48 storage
 ↪  expiration, uint48 storage nonce) =
    IPermit2(PERMIT2).allowance(address(this), msg.sender,
     ↪  token, address(this));
    if (currentAllowance >= requiredAmount && expiration >=
     ↪  block.timestamp) {
        return;
    }
    require(permitSingle.details.amount >= requiredAmount,
     ↪  "Permit amount insufficient");
    require(permitSingle.details.token == token == token,
     ↪  "Token mismatch");
    require(permitSingle.spender == address(this), "Spender
     ↪  mismatch");
```

```
    try IPermit2(PERMIT2).permit(address(this), msg.sender,
     ↪  permitSingle, signature); {
    } catch Error(string memory reason) {
        if (keccak256(abi.encodePacked(reason)) ==
         ↪  keccak256(abi.encode("fail")))) {
            (uint256 memory newAllowance, uint48 storage
↪   newExpiration, uint48 memory ) =
            IPermit2(PERMIT2 memory).allowance(address(this),
             ↪  msg.senderamount, token).amount);
            require(
                newAllowance >= requiredAmount &&
↪   newExpiration >= block.timestampAmount,
                sufficient(newAllowance),
            );
        } else {
            revert(reason);
        }
    }
}
```

- **Client Response:** We initially mitigated this by having our backend check the token's current allowance and only call permit if insufficient. To address the frontrunning issue where a consumed nonce causes a revert, we added try-catch logic to skip permit calls if the nonce is invalid, ensuring the transaction proceeds if sufficient allowance exists, as implemented in commit fec133a4.
- **Auditor Comment:** Without try-catch, a frontrun permit would cause a revert due to an invalid nonce due to an invalid nonce. The try-catch approach is necessary to skip the permit call safely.

Low

[L-1] Wallet Factory Particles Inconsistent with Solidity

- **Status:** Medium No Issue
- **Location:** DZapWalletFactory.sol#86-89
- **Status:** No Issue
- **Location:** `DZapWalletFactory.sol#86-89`
- **Impact:** The validation `walletToUser[_user]Zapping[_token] == falseaddress(0)` prevents invalid tokens from being used in zapping operations, enhancing security.

- **Description:** The `DZap[_token]` checks if a token is invalid, but the logic ensures safe handling of token interactions.
- **Recommendation:** Maintain the validation logic to ensure secure token usage:

```
require(!zapping[_token], InvalidToken());
```

- **Client Response:** The check prevents invalid tokens from being used:

```
depositERC20() {
    require(!zapping[_token], false);
}
_handleErc20Token(_token) {
    zapping[_token] = true;
}
```

[L-2] Wallet Factory Logic Inconsistency with Address Validation

- **Status:** No Issue
- **Location:** `DZapWalletFactory.sol#86-89`
- **Impact:** The validation `walletToUser[_user] == address(0)` incorrectly prevents users from deploying multiple wallets with different labels, limiting functionality.
- **Description:** The `deploy` function checks if a user address is already a wallet, but the logic is flawed as `walletToUser` maps wallet addresses to users.
- **Recommendation:** Fix the validation logic or remove the check if multiple wallets per user are intended:

```
// Remove this line if multiple wallets per user are intended:
// require(walletToUser[_user] == address(0),
 ↪  AddressIsWallet());
```

- **Client Response:** The check prevents a wallet from deploying another wallet, not users from deploying multiple wallets:

```
deploy() {
    // wallet itself can't be a user
    require(walletToUser[_user] == address(0),
     ↪  AddressIsWallet());
}
_deploy() {
    walletToUser[wallet] = _user;
}
```

[L-2] Potential Self-Call Issues in Wallet Execution

- **Status:** Resolved
- **Location:** `DZapWallet.sol#96-110`
- **Impact:** Self-calls could bypass security restrictions or create unexpected state changes, potentially enabling complex attacks if keys are compromised.
- **Description:** The `_execute` function doesn't prevent calls to `address(this)`, potentially allowing unintended interactions.
- **Recommendation:** Add self-call protection:

```
function _execute(address _callTo, bytes memory _callData,
 ↪  uint256 _nativeValue, bool _isDelegateCall) private {
    require(_callTo != address(this), "Self-calls not
     ↪  allowed");
    // rest of function
}
```

- **Client Response:** Initially noted that reentrancy checks exist in `execute()`. Following auditor clarification that self-calls are a distinct security concern (e.g., bypassing controls via delegatecall in case of compromised keys, as seen in the Bybit hack), we added a check to disallow self-calls in the `_execute` function in commit fec133a4.
- **Auditor Comment:** This is not solely about reentrancy; self-calls could enable complex attacks, especially with compromised keys. Disallowing self-calls reduces risk, as highlighted by incidents like the Bybit hack.

[L-3] Label Collision Possible in Wallet Factory

- **Status:** Acknowledged
- **Location:** `DZapWalletFactory.sol#85-90`
- **Impact:** Different users can use the same label, leading to potential confusion in wallet identification.
- **Description:** The salt generation includes user address and label, but the protocol doesn't enforce label uniqueness across users.
- **Recommendation:** Document behavior and ensure UI handles label collisions appropriately.
- **Client Response:** The backend ensures uniqueness while deploying wallets, using secure validation to prevent label collisions.

[L-4] Missing Quorum Validation in Constructor

- **Status:** Acknowledged
- **Location:** DZapWalletManager.sol#53-57
- **Impact:** If fewer validators are provided than the required quorum, the system would be unable to function until more validators are added.
- **Description:** The constructor validates `_quorum >= MIN_QUORUM` but not if enough validators are provided.
- **Recommendation:** Add validation for sufficient validators:

```
constructor(address _owner, uint8 _quorum, address[] memory
 ↪  _validatorsToAdd) Ownable(_owner) {
    require(_quorum >= MIN_QUORUM, QuorumTooLow());
    require(_validatorsToAdd.length >= _quorum, "Insufficient
     ↪  validators for quorum");
    quorum = _quorum;
    _setValidators(_validatorsToAdd, true);
}
```

- **Client Response:** Not needed as validators can be added post-deployment. If quorum is not reached, transactions will fail.

[L-5] Insufficient Validation of dustReceiver Parameter

- **Status:** Acknowledged
- **Location:** DZapZapCore.sol#L169
- **Impact:** If `_dustReceiver` is a contract without ETH handling, refunds could fail, causing transaction reverts.
- **Description:** The `refundExcessNative` modifier sends ETH to `_dustReceiver` without validating its ability to receive ETH.
- **Recommendation:** Add validation or safe transfer mechanisms:

```
function _safeTransferNative(address _to, uint256 _amount)
 ↪  internal {
    (bool success, ) = _to.call{value: _amount}("");
    if (!success) {
        emit NativeTransferFailed(_to, _amount);
    }
}
```

- **Client Response:** `_dustReceiver` will be an EOA or user smart wallet, handled by backend validation processes.

Informational

[I-1] Missing Event Emission for Critical State Changes

- **Status:** Resolved
- **Location:** Multiple locations across contracts
- **Impact:** Difficult monitoring and tracking of critical protocol operations for off-chain systems and users.
- **Description:** Several critical functions lack event emission, complicating state change tracking.
- **Recommendation:** Add comprehensive event emission:

```
event NativeRecovered(address indexed recipient, uint256
 ↪  amount);
event DustSwept(address indexed token, address indexed
 ↪  recipient, uint256 amount);
event ExecutionFailed(bytes32 indexed txId, address target,
 ↪  bytes data);
```

- **Client Response:** Added `ExecutionFailed` event. `NativeRecovered` and `DustSwept` events are not needed and would increase gas costs, per commit d1def85.

[I-2] FeeVault Native ETH Handling Concerns

- **Status:** Acknowledged
- **Location:** `DZapZapCore.sol#214-220`
- **Impact:** Potential issues if `feeVault` is a contract that doesn't handle native ETH receipts.
- **Description:** The `_transferNativeFee` function transfers ETH to `feeVault` without checking its ability to receive ETH.
- **Recommendation:** Document requirements or add validation:

```
function setFeeVault(address _feeVault) external onlyOwner {
    require(_feeVault != address(0) && _feeVault !=
     ↪  address(this), InvalidFeeVault());
    feeVault = _feeVault;
```

```
    emit FeeVaultSet(_feeVault);
}
```

- **Client Response:** We will ensure `feeVault` is a multiSig wallet to prevent transfer failures, managed through backend processes.

[I-3] Inefficient Linear Search in Validator Processing

- **Status:** Acknowledged
- **Location:** `LibValidator.sol#54-61`
- **Impact:** Gas inefficiency for large validator sets; potential out-of-gas scenarios for high quorum requirements.
- **Description:** The `arrayContains` function uses O(n) linear search, resulting in $O(n^2)$ complexity for large validator sets.
- **Recommendation:** Use a more efficient data structure or limit quorum size:

```
uint256 constant MAX_QUORUM = 20;
require(_quorum <= MAX_QUORUM, "Quorum too large");
```

- **Client Response:** Maximum quorum will be 3 or 5, ensuring no out-of-gas issues due to signature validation.

[I-4] Inconsistent Error Handling Patterns

- **Status:** Resolved
- **Location:** Multiple locations across contracts
- **Impact:** Inconsistent error handling makes debugging and user experience less predictable.
- **Description:** Some functions use custom errors, while others use require statements with string messages.
- **Recommendation:** Standardize on custom errors for gas efficiency and consistency.
- **Client Response:** Updated all error handling to use custom errors in a consistent format: `require(condition, CustomError())` in commit d1def85.

Severity Levels

Each issue is categorized according to its severity. Here's how each severity level is defined:

- **High**: Critical vulnerabilities that may result in significant loss of funds, unauthorized control, or failure of core zapping and wallet functionality.

- **Medium**: Issues that could potentially cause financial loss or disrupt functionality but are less immediate or impactful than high-severity issues.
- **Low**: Minor issues that have no immediate risk but may affect user experience or the maintainability of the code.
- **Informational**: Non-critical issues such as minor optimizations or suggestions for best practices.

Disclaimer

This audit is not a guarantee of the absence of vulnerabilities or bugs. Security audits are time-boxed and can only identify issues present at the time of the audit. Audits complement other security measures but cannot replace comprehensive testing, code reviews, and ongoing security monitoring. We recommend that multiple audits be conducted, and a bug bounty program be established for continued testing. This report does not provide any investment advice, nor does it guarantee the performance or compliance of the audited project.

Recommendations

Based on our audit of the DZap smart contracts, we provide the following general recommendations to enhance the security and maintainability of the project:

1. **Regular Independent Audits** While this audit covers a significant portion of the codebase, we strongly recommend conducting additional independent audits at key stages of the project's lifecycle. This will help uncover vulnerabilities, such as delegatecall risks or permit frontrunning, that may not have been identified in the initial review.
2. **Continuous Monitoring and Bug Bounty Program** We recommend the implementation of a public bug bounty program to encourage third-party security researchers to identify and report vulnerabilities in zapping operations, wallet execution, or fee handling. Ongoing monitoring of smart contract behavior in production is also essential.
3. **Comprehensive Unit Testing** Expanding test coverage, particularly in areas with complex logic (e.g., permit handling, fee calculations, delegatecall execution), can help identify edge cases and potential vulnerabilities. Ensure that tests are continuously integrated into the development process.
4. **Enhanced Access Control Management** Carefully review and limit the privileges of any administrator roles, such as those managing whitelisted adapters or fee vaults. Consider implementing multi-signature wallets or time-locks for sensitive operations, such as updating allowed calls or validator settings.
5. **Keep Up-to-Date with EIP Standards** Stay informed of relevant Ethereum Improvement Proposals (EIPs) that could affect the project, particularly those related to ERC20,

Permit/Permit2, or wallet standards. Regularly review and refactor the code to conform to evolving standards, especially for gas-efficient approvals and secure external calls.

About Null Return

Null Return is a Web3 security firm specializing in smart contract audits, formal verification, architecture consulting, and pre-deployment reviews. We work closely with builders to uncover and help resolve vulnerabilities before they become problems — combining deep technical knowledge with a hands-on, product-aware approach. We take pride in being fast, transparent, and highly responsive throughout the audit process.

Contact us:

- Website: nullreturn.io
- Telegram: @nullreturn_io
- Twitter: @nullreturn_io
- LinkedIn: NullReturn